

✖ Java Exception Handling Assignments

✓ Assignment 1: `throw` and `throws`

1. Throwing `IllegalArgumentException`

Goal: Understand how to manually throw an exception based on input.

Task:

- Create a method named `checkPositive` that takes an integer input.
- If the input is negative, throw `IllegalArgumentException` with a meaningful message.
- Otherwise, print "Valid number".

💡 **Example:** `checkPositive(-5); // Should throw:
IllegalArgumentException("Number cannot be negative")`

2. `throws` keyword with file reading

Goal: Learn to declare exceptions using `throws`.

Task:

- Write a method named `readFile` that takes a filename as input.
- Declare it with `throws IOException`.
- Inside, use `BufferedReader` and `FileReader` to read and print the file content.
- Don't catch exceptions inside the method. Catch them in `main()`.

💡 **Example:** `readFile("nonexistent.txt"); // Should throw
IOException, caught in main`

✓ Assignment 2: Checked vs Unchecked Exceptions

1. Classify exceptions

Goal: Understand which exceptions are checked and which are unchecked.

Task:

Given these exceptions:

- `IOException`
- `SQLException`
- `NullPointerException`
- `ArithmeticException`
- `FileNotFoundException`

Classify each one and explain:

Hint:

- **Checked:** Must be handled or declared using `throws`.
- **Unchecked:** Don't require handling (subclass of `RuntimeException`).

2. Handle checked exceptions

Goal: Learn to use `try-catch` for checked exceptions.

Task:

- Write a method named `loadFile` that opens a file using `FileInputStream`.
- Catch `FileNotFoundException` and `IOException`.
- Print clear error messages.
- Ensure the file stream is closed properly.

 **Example:** `loadFile("test.txt");` // Should handle file not found errors

✓ Assignment 3: Custom Exceptions

1. Define a custom exception

Goal: Learn to define your own exception class.

Task:

- Create a class named `LoginFailedException` that extends `Exception`.
- Add a constructor that accepts a string message and passes it to `super()`.

 **Example:** `throw new LoginFailedException("Invalid login details");`

2. Use your custom exception

Goal: Practice using and handling custom exceptions.

Task:

- Create a method `login(String username, String password)`.
- If the input doesn't match expected credentials, throw `LoginFailedException`.
- In `main()`, take user input, call `login()`, and catch the exception.

 **Example:** `login("xyz", "123"); // Should throw LoginFailedException`

✓ Assignment 4: Try-With-Resources

1. File reading with try-with-resources

Goal: Learn how to auto-close resources using try-with-resources.

Task:

- Write a method named `readFileSafely` using `BufferedReader` and `FileReader` inside a try-with-resources block.
- Read and print file contents line-by-line.
- Catch `IOException` and print a message.

 **Example:** `readFileSafely("sample.txt");`

2. Reading Input Using Scanner with try-with-resources

Goal: Read from a file using `Scanner` inside try-with-resources.

Task:

- Create a method `scanFile(String fileName)`.
- Use `Scanner` in a try-with-resources block.
- Print each line read from the file.

 **Example:** `scanFile(String fileName) // Use try-with-resources to handle Scanner for file access`

✓ Assignment 5: BankATM with Scanner Input – Handle Insufficient Balance

Goal:

Use custom exceptions.

Take user input using `Scanner`.

Apply `try-catch` around business logic like balance validation.



Task:

- Create `InsufficientBalanceException` class.
- Create `BankATM` class with:
 - `deposit(double amount)`
 - `withdraw(double amount)` (throws custom exception)
 - `getBalance()`
 - In `main()`:
 - ❖ Use `Scanner` to allow user to choose deposit or withdraw.
 - ❖ Continue prompting until the user exits.
 - ❖ Catch and display messages for `InsufficientBalanceException`.



Example:

```
// Step 1: Custom Exception
class InsufficientBalanceException

// Step 2: ATM Logic
class BankATM : deposit, withdraw, getBalance

// Step 3: User Input via Scanner

class ATMTTest : main(), Scanner, switch(action)
```
